

第二章 软件开发过程模型

北京邮电大学
网络空间安全学院
芦效峰



软件开发过程模型

本章要点

- 一、软件生存周期
- 二、常用软件开发过程模型

2.1 软件生存周期

一个软件从定义、开发、使用、维护，直到最终被废弃，要经历一个漫长的时期，这就如同一个人要经过胎儿、儿童、青年、中年、老年，直到最终死亡的漫长时期一样，通常把软件经历的这个漫长时期称为生存期。

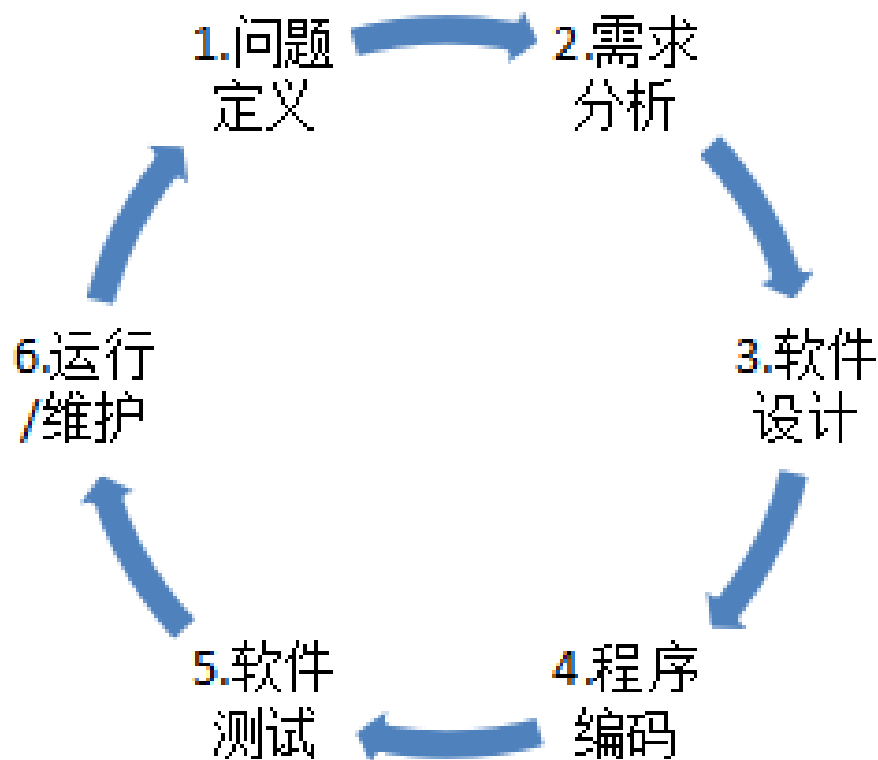


软件生存期 life cycle

- 软件生存期的六个活动，即可行性研究、需求分析、设计、程序编码、测试及运行维护
- 软件工程学的一个重要目标就是提高软件的可维护性，减少软件维护的代价。
- 把整个软件生存周期划分为若干阶段，使得每个阶段有明确的任务，使规模大、结构复杂和管理复杂的软件开发变得容易控制和管理。

软件生存周期

软件产品经历了从开始到结束的整个开发周期后，新一代产品将通过开发周期的重复而发展。



问题定义和可行性研究

- 回答：系统要解决什么问题，是否可行？
- 确定要开发软件系统的总目标及可行性
- 给出功能、性能、可靠性以及接口等方面的要求
- 估计可利用的资源（硬件，软件，人力等）、成本、效益、开发进度
- 制定出完成开发任务的实施计划，连同可行性研究报告，提交管理部门审查

需求分析和定义

- 对用户提出的要求进行**分析**并给出**详细的定义**
- **不是解决问题，而是确定软件系统必须做什么，及详细定义**
- **编写软件需求说明书或系统功能说明书及初步的系统用户手册**
- **提交管理机构评审**

软件设计

- 把需求转变为软件结构，及模块的功能描述
- 概要设计 – 把各项需求转换成软件的体系结构。结构中每一组成部分都是意义明确的模块，每个模块都和某些需求相对应
- 详细设计 – 对每个模块要完成的工作进行具体的描述，为源程序编写打下基础
- 编写设计说明书，提交评审。

程序编写

- 把软件设计转换成计算机可以接受的程序代码，即写成以某一种特定程序设计语言表示的“源程序清单”
- 要求：写出的程序应当是结构良好、清晰易读的，且与设计相一致的，并且是安全的
- 大部分计算机课程在这一层

软件测试

- 发现存在的问题
- 单元测试、集成测试、确认测试、系统测试几个阶段
- 单元测试，查找各模块内部在功能和结构上存在的问题并加以纠正
- 集成测试，将已测试过的模块按一定顺序组装起来
- 确认测试，按规定的各项需求，逐项进行有效性测试，决定已开发的软件是否合格，能否交付用户使用



运行 / 维护

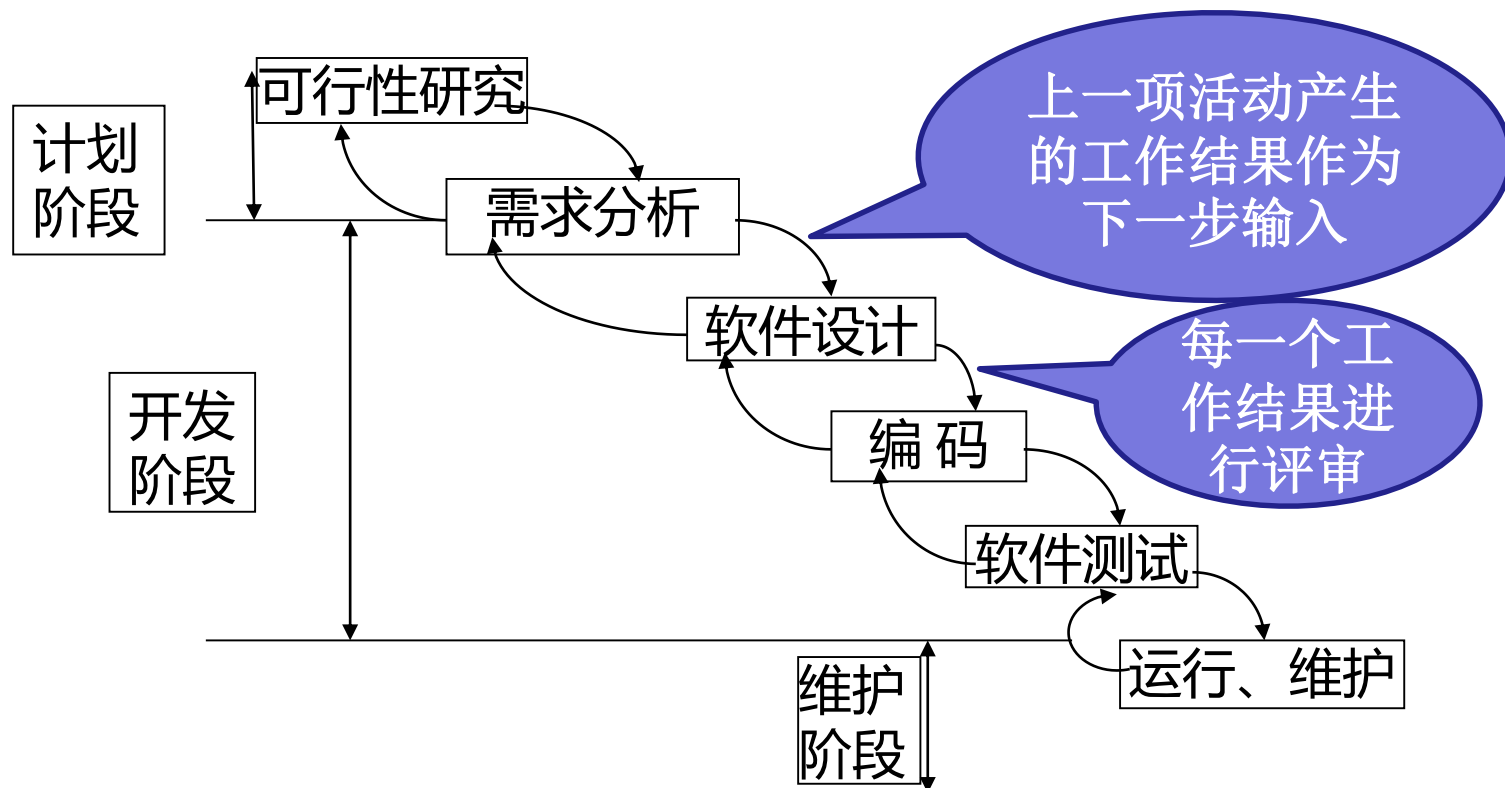
修改在运行中发现的软件错误，完善功能，或为了适应变化了的软件工作环境，需做适当变更。

软件开发模型

- 软件生存期内各种活动的组织与衔接都是可以由软件开发组织规定的。
- 为了研究软件开发项目中各种活动的一般规律，以及对软件开发过程进行定量度量 and 优化，人们提出了软件开发过程模型。
- 描述软件从软件需求定义到软件交付使用后到报废为止，各种活动如何执行的模型。

2.2 瀑布模型

将各项活动规定为依固定顺序连接的若干阶段工作，
从上到下，不可逆转。以文档为驱动

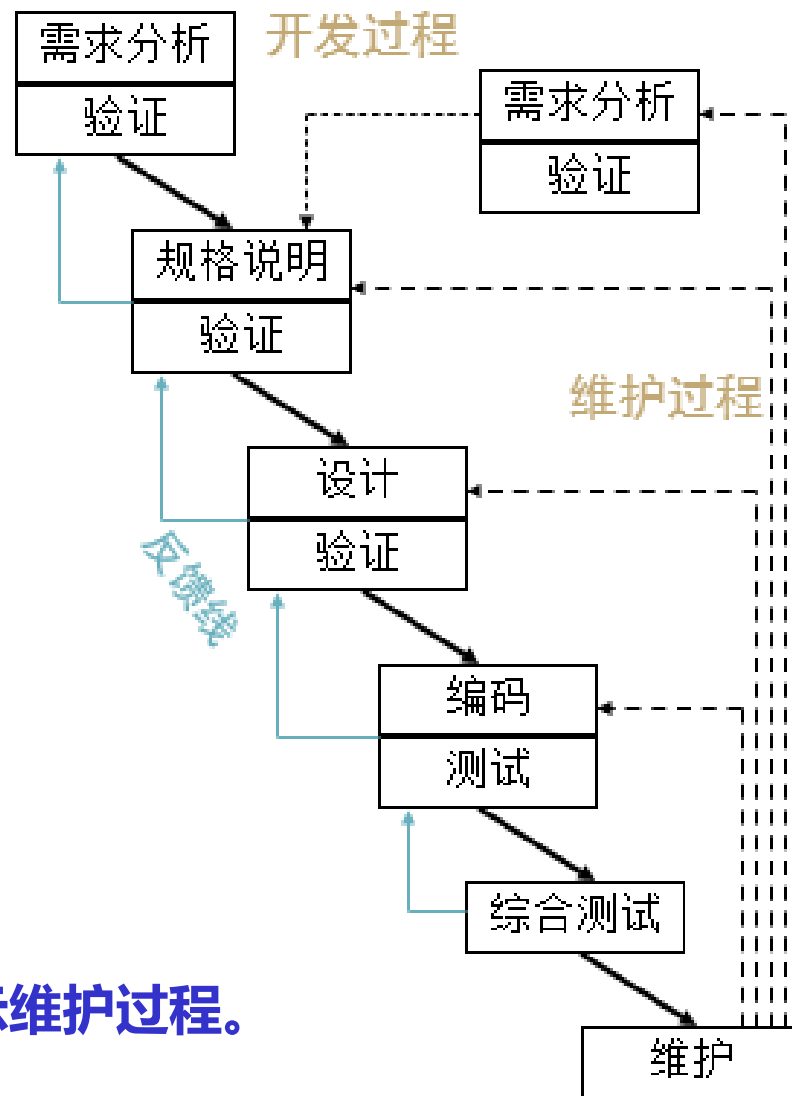


瀑布模型特点

按照传统的瀑布模型来开发软件，有以下特点：

- 1、阶段间有顺序性和依赖性；
- 2、推迟实现的观点；
- 3、质量保证的观点；

优点：可强迫开发人员采用规范的方法；严格规定了每个阶段必须提交的文档；要求每个阶段交出的产品都必须经过质量保证小组的仔细验证。



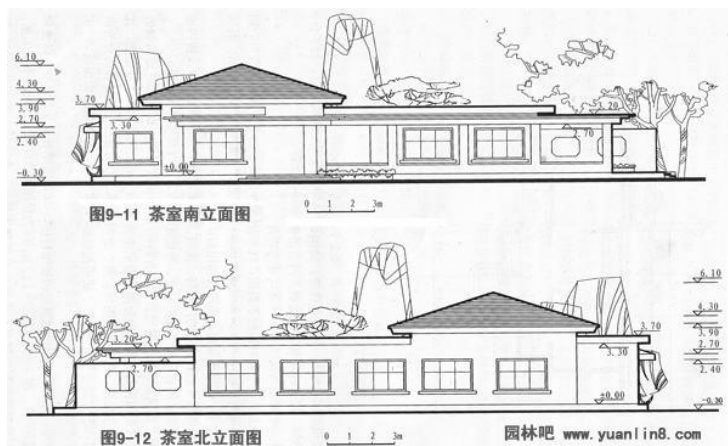
实线箭头表示开发过程；虚线箭头表示维护过程。

瀑布模型缺点

- 项目开始阶段，开发人员和用户对需求的描述常常不全面。如果需求阶段没发现问题，则影响后面各阶段的工作
- 各阶段所做的工作都是文档说明，**一般用户不易理解文字所叙述的软件**。用户的修改意见加大了修改难度
- 改变会影响整个软件开发。事先选择的技术或需求迅速发生变化，需要返回到前面，对前面一系列内容进行修改。
- 案例

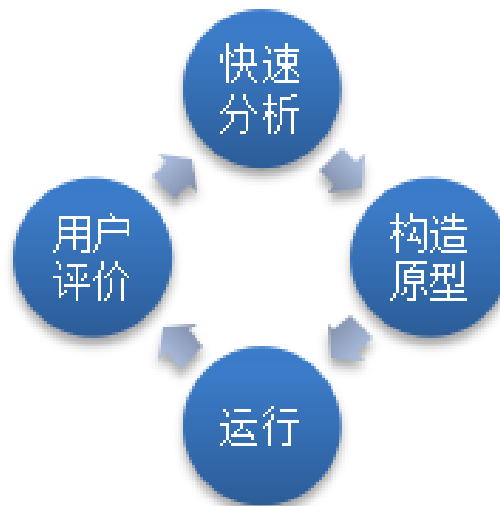
2.3 原型模型

- 由于在项目开发的初始阶段人们对软件的需求认识常常不够清晰，因而使得开发项目难以做到一次开发成功，出现返工再开发在所难免。
- 例子



2.3 原型模型

- 原型模型的第一步是建造一个快速原型，实现客户或未来的用户与系统的交互，用户或客户对原型进行评价，进一步细化待开发软件的需求。通过逐步调整原型使其满足客户的要求，开发人员可以确定客户的真正需求是什么；
- 第二步则在第一步的基础上开发客户满意的软件产品。



步骤

- 做两次或多次
- 第一次只是试验开发，其目标只是在于探索可行性，弄清软件需求
- 第二次则在此基础上获得较为满意的软件产品
- 显然，快速原型方法可以克服瀑布模型的缺点，减少由于软件需求不明确带来的开发风险，具有显著的效果。

模型的关键

- 快速原型的关键在于尽可能快速地建造出软件原型，一旦确定了客户的真正需求，**所建造的原型将被丢弃**。重要的是必须迅速建立原型，随之迅速修改原型，以反映客户的需求。
- 快速原型的开发途径
 - 1、仅模拟软件系统的人机界面和人机交互方式。
 - 2、开发一个工作模型，实现软件系统中重要的或容易产生误解的功能。
 - 3、利用一个或几个类似的正在运行的软件向用户展示软件需求中的部分或全部功能。
 - 4、绘图

原型模型的优点

- 快速实现的初步模型，可以供开发人员和用户进行交流，以便较准确获得用户的需求，可以澄清模糊需求，及早暴露系统实施后潜在的一些问题；
- 它可以大大避免在瀑布模型冗长的开发过程中看不见产品雏形的现象，可调动用户积极参与。
- 原型系统可作为培训环境，有利于用户培训和开发同步，开发过程也是学习过程。
- 总的开发费用低，时间缩短，开发效率提高。

原型模型的缺陷

- 1) 建立原型模型的软件工具与环境与实际模型的存在脱节的现象。
- 2) 以目前通用的开发工具，开发原型本身并不是件容易的事情。
- 3) 原型模型对用户深层次的需求并不能深入分析。
- 4) 关注当前需求，产品原型在一定程度上限制了开发人员的创新，没有考虑软件的整体质量和长期的可维护性。
- 5) 开发人员很可能采用一个不合适的操作系统或程序设计语言，仅仅因为它通用和熟悉；可能使用一个效率低的算法。经过一段时间之后，这些不理想的选择就成了系统的组成部分。

局限性

- 快速建立起来的系统结构加上连续的修改可能会导致产品质量低下；开发人员可能对低效或临时的选择已经习以为常，于是，这些不理想的选择就成了系统的组成部分。
- 由于达不到质量要求产品可能被抛弃，因此原型实现模型不适合嵌入式、实时控制及科学数值计算等大型软件系统的开发。

2.4 迭代模型

- 迭代式开发允许在每次迭代过程中需求可能有变化，通过不断细化来加深对问题的理解。
- 早在20世纪50年代末期，软件领域中就出现了迭代模型。最早的迭代过程可能被描述为“分段模型（stagewise model）”。
- 在某种程度上，开发迭代是一次完整地经过所有工作流程的过程：需求分析、设计、实施和测试工作流程。
- 美国国防部推荐

案例：

- 假设某软件公司要开发一个软件，该软件共有A、B、C、D、E五个功能，但是用户对这五个功能只有基本的功能需求，其他需求仍然不是非常明确，该怎么开发呢？
- 迭代模型开发

与传统的瀑布模型相比较，迭代过程具有以下优点：

- 1) 降低了在一次迭代上的开发风险。
- 2) 降低了产品无法按照既定进度进入市场的风险。
- 3) 加快了整个开发工作的进度。因为开发人员清楚问题的焦点所在，他们的工作会更有效率。
- 4) 由于用户的需求并不能在一开始就作出完全的界定，它们通常是在后续阶段中不断细化的。因此，迭代过程这种模式使适应需求的变化会更容易些。

2.5 增量模型

- 增量模型是把待开发的软件系统模块化，将每个模块作为一个增量组件，从而分批次地分析、设计、编码和测试这些增量组件。运用增量模型的软件开发过程是一种递增式的过程。
- 假设某软件公司要开发一个软件系统，该系统共有A、B、C、D、E五个子系统，目前对于A、B、C三个子系统的需求比较明确，但是D、E还需要用户确认需求，该怎么开发呢？

工作流程

- 增量模型的工作流程分成三个阶段：
- 1、在系统开发的前期阶段，为了确保所建系统具有优良的结构，仍需要针对整个系统进行需求分析和概要设计，需要确定系统的基于增量组件的需求框架，并以需求框架中组件的组成及关系为依据，完成对软件系统的体系结构设计。
- 2、在完成软件体系结构设计之后，可以进行增量组件的开发。这个时候，需要对组件进行需求细化，然后进行设计、编码测试和有效性验证。
- 3、在完成了对某个增量组件的开发之后，需要将该组件集成到系统中去，并对已经发生了改变的系統重新进行有效性验证，然后再继续下一个增量组件的开发。

增量模型优点

- 1、人员分配灵活，刚开始不用投入大量人力资源；
- 2、将待开发的软件系统模块化，可以分批次地提交软件产品，使用户可以及时了解软件项目的进展。
- 3、开发顺序灵活。开发人员可以对组件的实现顺序进行优先级排序，先完成需求稳定的核心组件。当组件的优先级发生变化时，还能及时地对实现顺序进行调整。
- 4、以组件为单位进行开发降低了软件开发的風險。一个开发周期内的错误不会影响到整个软件系统。

增量模型的缺点

- 如果增量包之间存在相交的情况且未很好处理，则必须做全盘系统分析，这种模型将功能细化后分别开发的方法较适应于需求经常改变的软件开发过程。

2.6 螺旋模型

对于复杂的大型软件，开发一个原型往往达不到要求。1988年，Barry提出螺旋模型，将瀑布模型和原型结合，注重风险分析，适合于大型复杂系统。

软件开发的风险：

- 1、 产品交付用户之后用户可能不满意；
- 2、 到交付日期后软件可能还未开发出来；
- 3、 实际开发成本可能超过预算；
- 4、 一些关键的开发人员可能“跳槽”；
- 5、 聘请不到需要的专业人员；

基本思想：使用原型及其他方法来尽量降低风险。可看作在每个阶段之前都增加了风险分析过程的快速原型模型。螺旋线每个周期对应一个开发阶段。

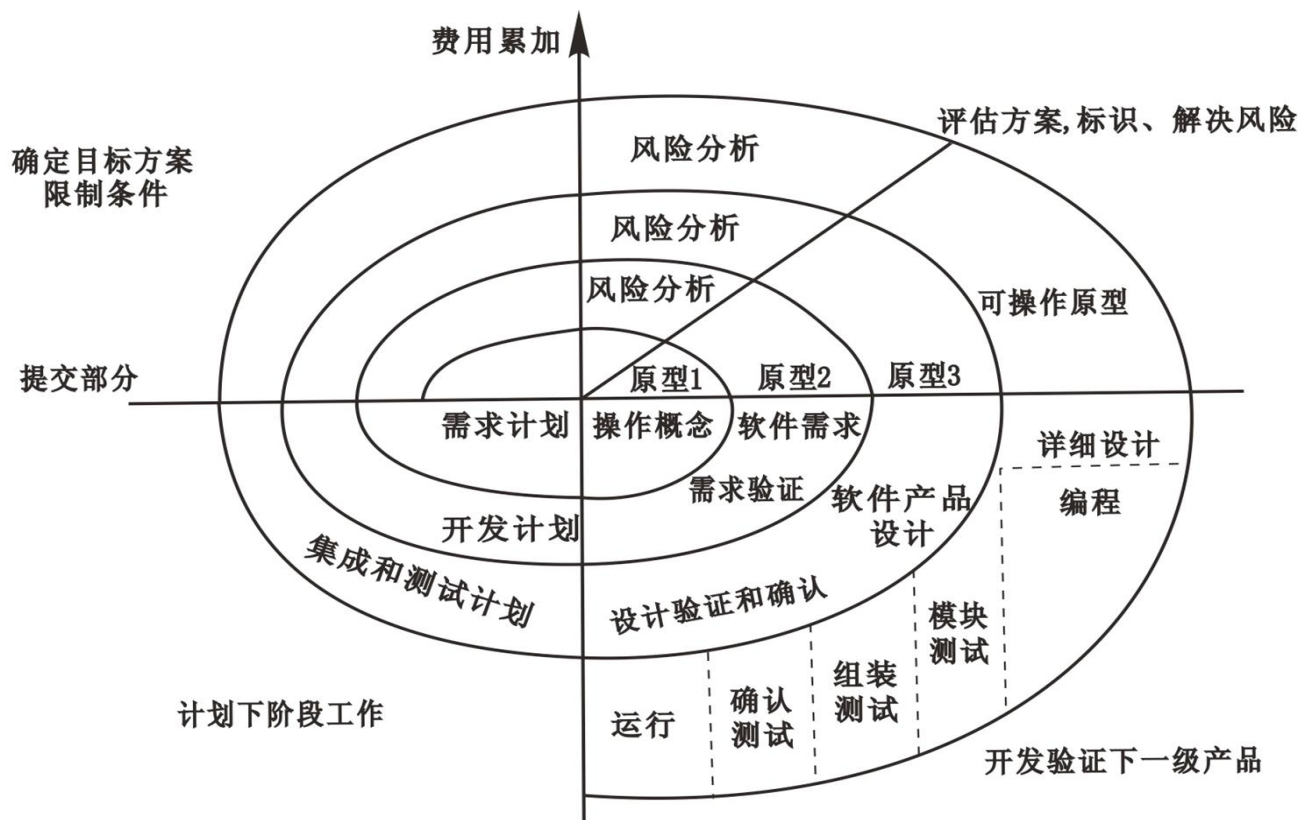
螺旋模型

螺旋模型沿着螺线旋转，在四个象限上分别表达四个方面的活动，即：

- **制定计划**——确定软件目标，选定实施方案，弄清项目开发的限制
- **风险分析**——分析所选方案，考虑如何识别和消除风险
- **实施工程**——实施软件开发
- **客户评估**——评价开发工作，提出修正建议

螺旋模型

沿着螺旋线每转一圈，表示开发一个更完善的版本。



螺旋模型

1. 每个阶段开始时（左上象限）的任务是：确定该项目的目标、为完成这些目标选择方案及设定这些方案的约束条件。
2. 接下来的任务是：从风险角度分析上步的工作结果，努力排除各种潜在的风险，如果风险不能排除，则停止开发工作或大幅度削减项目规模。
3. 如果成功排除了所有风险，则启动下一个步骤（右下象限），这个步骤相当于纯粹的瀑布模型。
4. 最后是评价该阶段工作成果并计划下一阶段的工作。

螺旋模型

特点

“螺旋模型”的核心就在于不需要在刚开始的时候就把所有事情都定义的清清楚楚。轻松上阵，定义最重要的功能，实现它，然后听取客户的意见，之后再进入到下一个阶段。如此不断轮回重复，直到得到满意的最终产品。

螺旋模型

优点:

风险驱动的，每个方案在实施前都要经过风险分析。如果风险过大，则项目应该停止，或改变方案。减少了过多的测试或测试不足所带来的风险。

缺陷:

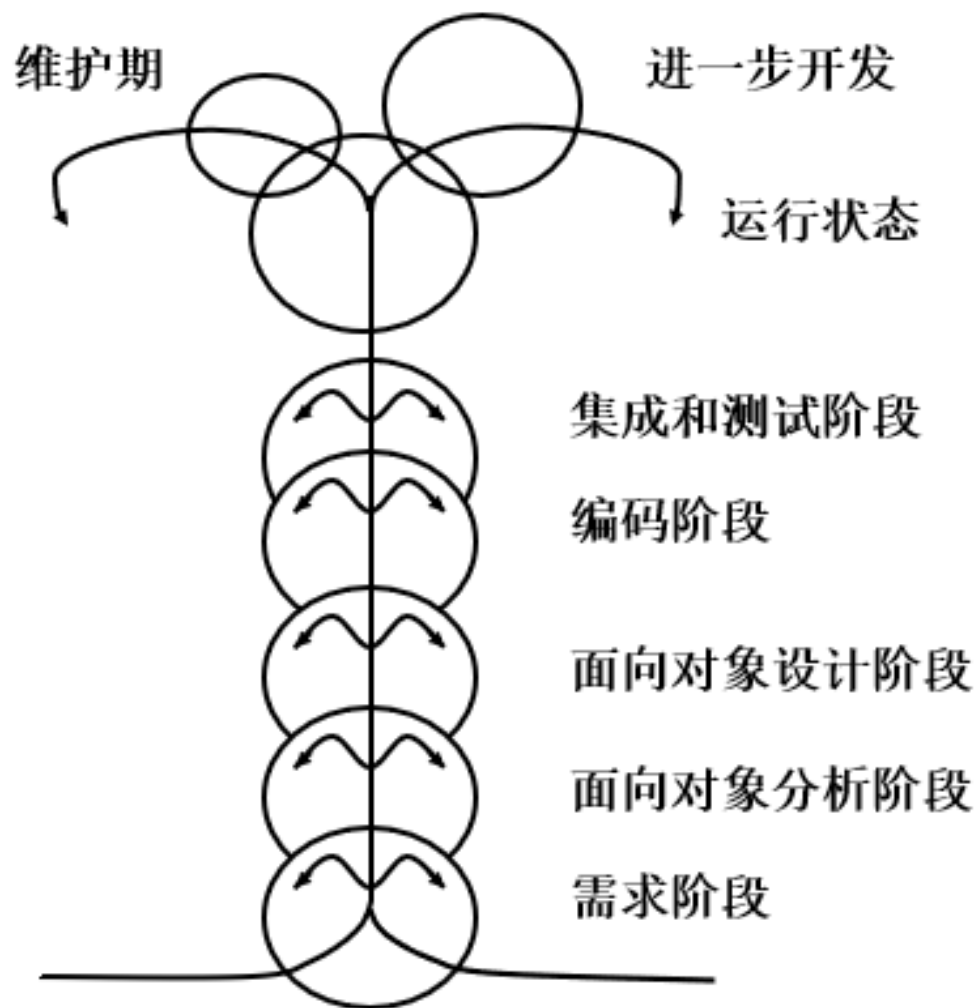
- 1) 采用螺旋模型需要具有相当丰富的风险评估经验和专门知识，在风险较大的项目开发中，如果未能够及时标识风险，势必造成重大损失。 以及客户认可。
- 2) 过多的迭代次数会增加开发成本，延迟提交时间。
- 3) 如果执行风险分析会大大影响项目的利润，那么风险分析就没有意义了。只适合大规模软件开发。

2.7 喷泉模型

- 喷泉模型是一种以用户需求为动力，以对象为驱动的模式，主要用于描述面向对象的软件开发过程。
- 该模型认为软件开发过程自下而上周期的各阶段是相互重叠和多次反复的，就像水喷上去又可以落下来，类似一个喷泉。
- 各个开发阶段没有特定的次序要求，并且可以交互进行，可以在某个开发阶段中随时补充其他任何开发阶段中的遗漏。

喷泉模型

各个开发阶段没有特定的次序要求，可以随时补充其他任何开发阶段中的遗漏。



喷泉模型

面向对象项目

- 喷泉模型主要用于面向对象的软件项目，软件的某个部分通常被重复多次，相关对象在每次迭代中随之加入渐进的软件成分。各活动之间无明显边界，例如设计和实现之间没有明显的边界，这也称为“喷泉模型的无间隙性”。
- 由于对象概念的引入，表达分析、设计及实现等活动只用对象类和关系，从而可以较容易地实现活动的迭代和无间隙。

喷泉模型特点

- 1、喷泉模型规定软件开发过程有4个阶段，即分析、系统设计、软件设计和实现。
- 2、各阶段相互重叠，表明了面向对象开发方法各阶段间的交叉和无缝过渡。
- 3、喷泉模型以分析为基础，资源消耗成塔形，在分析阶段消耗的资源最多。
- 4、喷泉模型反映了软件过程迭代性的自然特性。
- 5、喷泉模型强调增量开发，不要求一个阶段的彻底完成，整个过程是一个迭代的逐步提炼的过程，即做到边分析，边设计；边实现，边测试，使相关功能随之加入到演化的系统中。
- 6、喷泉模型是对象驱动的过程，对象是所有活动作用的实体，也是项目管理的基本内容。
- 7、该模型很自然地支持软件的重用。

喷泉模型

缺点

- 由于喷泉模型在各个开发阶段是重叠的，因此在开发过程中需要大量的开发人员，因此不利于项目的管理。
- 此外这种模型要求严格管理文档，使得审核的难度加大，尤其是面对可能随时加入各种信息、需求与资料的情况。

2.8 组件集成模型

- 我们将一个或多个相关类的组合称为一个组件或构件。
- 组件集成模型融合了螺旋模型的许多特征，本质上是演化型的，开发过程是迭代的。
- 以前项目中创建的组件被存储在一个组件库中。一旦开发人员经过与用户交谈，确定软件目标和方案，就可以标识出所需组件。**首先搜索已有的组件库**，如果需要的组件已存在，就从库中提取出来复用。如果不存在，就采用面向对象方法开发它。

组件集成模型

组件集成模型的特点：

- 软件复用，软件复用最大的优势是减少了需要开发的软件数量，加快了软件交付，从而降低了开发成本，同时也降低了开发风险。
- 据QSM联合报告的公司称：组件组装缩短70%的开发周期，降低84%的项目成本。
- 如果软件业在若干年内逐步建立完善构件标准，那么组件集成模型将是实现软件生产规模化、工程化的一个最有前途的模型。

2.9 敏捷开发过程

- 敏捷不是一个过程，是一类过程的统称，它们有一个共性，就是符合敏捷价值观，遵循敏捷的原则。
- 敏捷方法告诉我们，开发项目是一个学习的体验，没有谁能完全理解所有需求之后才开始项目，即使是顾客也一样。顾客一开始有一些主意，但是他们也在项目的进展过程中更多和更深入地了解 and 明确自己的需要。同样，开发人员在一开始学习到他们能知道的东西，但是他们仍需要通过项目来学习更多的东西。

敏捷开发特点

- 敏捷就是“快”，快才可以适应目前社会的快节奏。要快，就要更多地发挥个人的个性思维。
- 敏捷开发是一种以人为核心、迭代、循序渐进的开发方法。
- 敏捷方法的两大主要特征是对“适应性”的强调与对“人”的关注。
- 强调对变化的快速响应能力，通过引入迭代式的开发手段。
- 主要驱动核心是人，与流水线开发的思维相左。

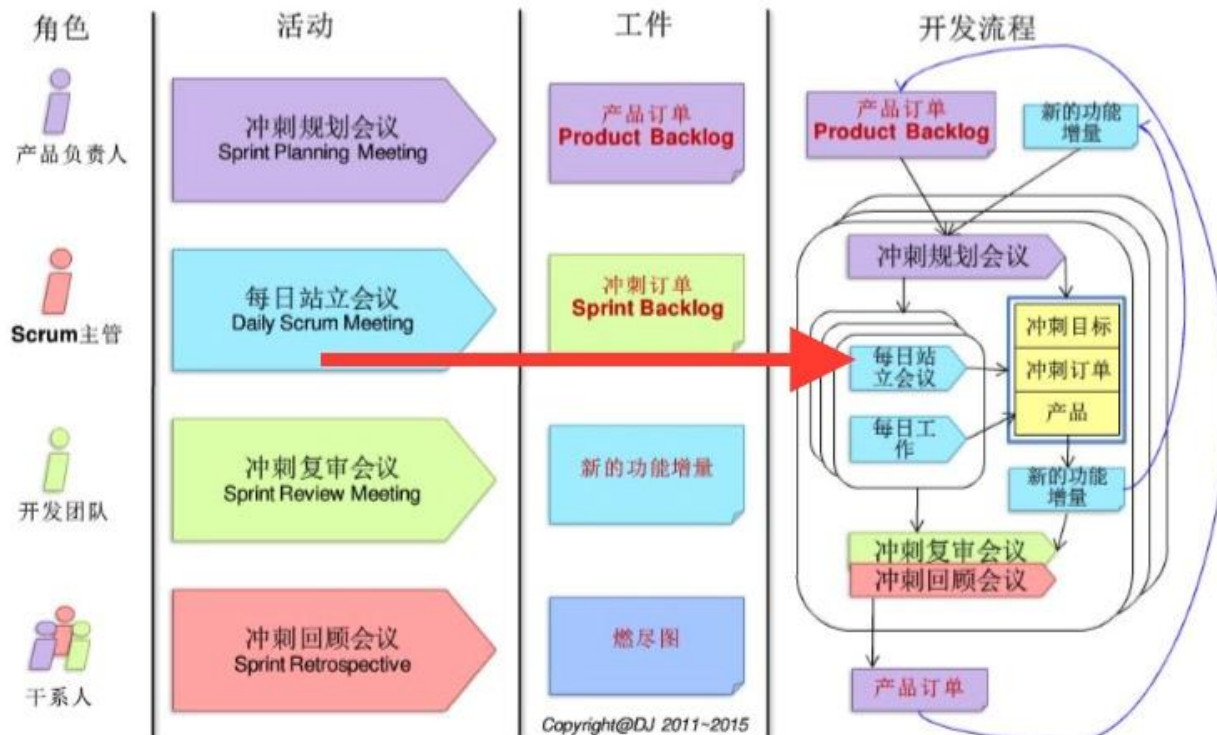
敏捷开发原则

- 1、**尽早并持续**地交付有价值的软件以满足顾客需求；
- 2、敏捷流程**欢迎需求的变化**，并利用这种变化来提高用户的竞争优势；
- 3、**经常发布**可用的软件，发布间隔可以从几周到几个月，能短则短；
- 4、**业务人员和开发人员**在项目开发过程中应该每天共同工作；
- 5、以有进取心的**人为项目核心**，充分支持信任他们；
- 6、无论团队内外，**面对面的交流**始终是最有效的沟通方式；

敏捷开发原则

- 7、可用的软件是衡量项目进展的主要指标；
- 8、敏捷流程应能保持可持续的发展，领导、团队和用户应该能按照目前的步调持续合作下去；
- 9、只有持续关注技术和设计，才能越来越敏捷；
- 10、保持简明（尽可能简化工作量的技艺）极为重要；
- 11、只有能自我管理的团队才能创造优秀的架构、需求和设计；
- 12、时时总结如何提高团队效率，并付诸行动。

Scrum敏捷开发流程



每日站立会议



任务看板



特点：短周期迭代

迭代是指把一个复杂且开发周期很长的开发任务，分解为很多小周期可完成的任务，这样的一个小周期就是一次迭代的过程；同时每一次迭代都可以生产或开发出一个可以交付的软件产品。

2.10 微软开发方法

- 《微软的秘密》对微软公司软件产品开发过程进行了介绍。
- 三个阶段：计划阶段、开发阶段、稳定化阶段
 - (1) 计划阶段
- 产生：想象性描述、产品说明，接口标准、最初的测试计划、文档策划、市场营销
- 想象性描述主要定义产品开发目标，不涉及具体细节。

微软开发方法

- 想象性描述包括对竞争对手的产品分析、未来版本的规划、必须解决的问题，新功能。
- 产品说明：定义出新的产品特性，并赋予不同优先级，及特性之间的关系。随着项目深入，不断添加细节。
- 对重要产品的分析和设计要高层领导复审。
- 基于产品说明，管理部门指定进度表，安排开发组。

微软开发方法

(2) 开发阶段

- 首先做计划，定里程碑版本，产品分成3、4个子项目，测试人员与开发人员配对，不断测试
- 基于产品说明开发。
- 任务细化到4小时~3天
- 预留1/3的缓冲时间

(3) 稳定化阶段

- 着重对产品测试和调试，不再增加新的功能
- 内部测试和外部测试

2.11 软件安全开发生命周期

- 1 什么是软件安全开发
- 软件安全开发涵盖了软件的整个生命周期，安全软件开发生命周期 (Secure Software Development Lifecycle, SSDL) 是一种强调安全措施的软件生命周期，即通过软件开发的各个步骤来确保软件的安全性，其目标是最大限度地确保软件的安全。

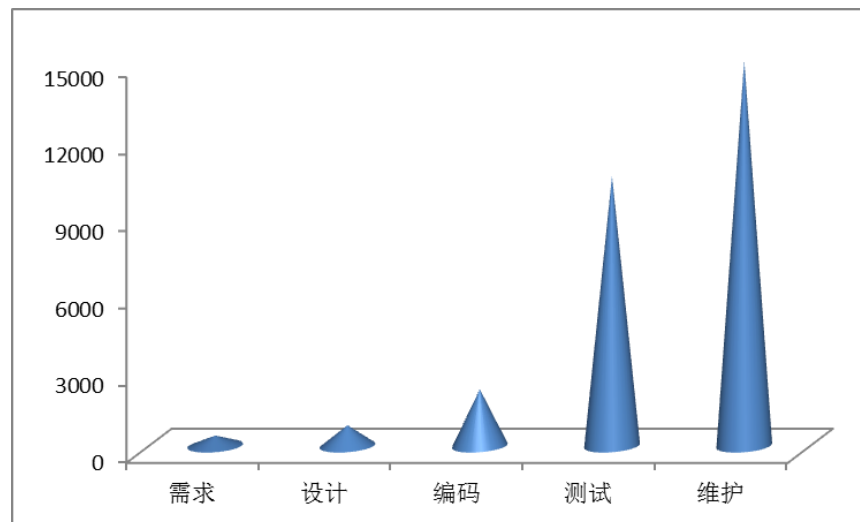
2.11 软件安全开发生命周期

- 软件安全开发过程在传统软件开发过程的各个阶段添加了安全措施和安全手段，防止因设计、开发、提交、升级或维护中的缺陷而产生不该有的软件漏洞和脆弱性。
- 在需求分析阶段就开始考虑软件的安全需求问题，在设计阶段设计符合安全需求的安全功能，在编码阶段保证开发的代码符合安全编码规范，并通过安全测试，确保安全需求、安全设计、安全编码各个环节得以正确、有效实施。
- 对软件在需求和设计阶段存在的安全问题，软件测试发现不了。

软件安全开发生命周期

降低解决软件安全问题的代价

- 早期发现并改正错误有以下优点
 - 在软件开发生命周期中，后面的阶段改正错误开销比前面的阶段要高出数倍
 - 早期发现并改正错误可以减少时间和开销



2 安全开发生命周期

- 可信计算安全开发生命周期SDL

微软, 2002.1, 盖茨

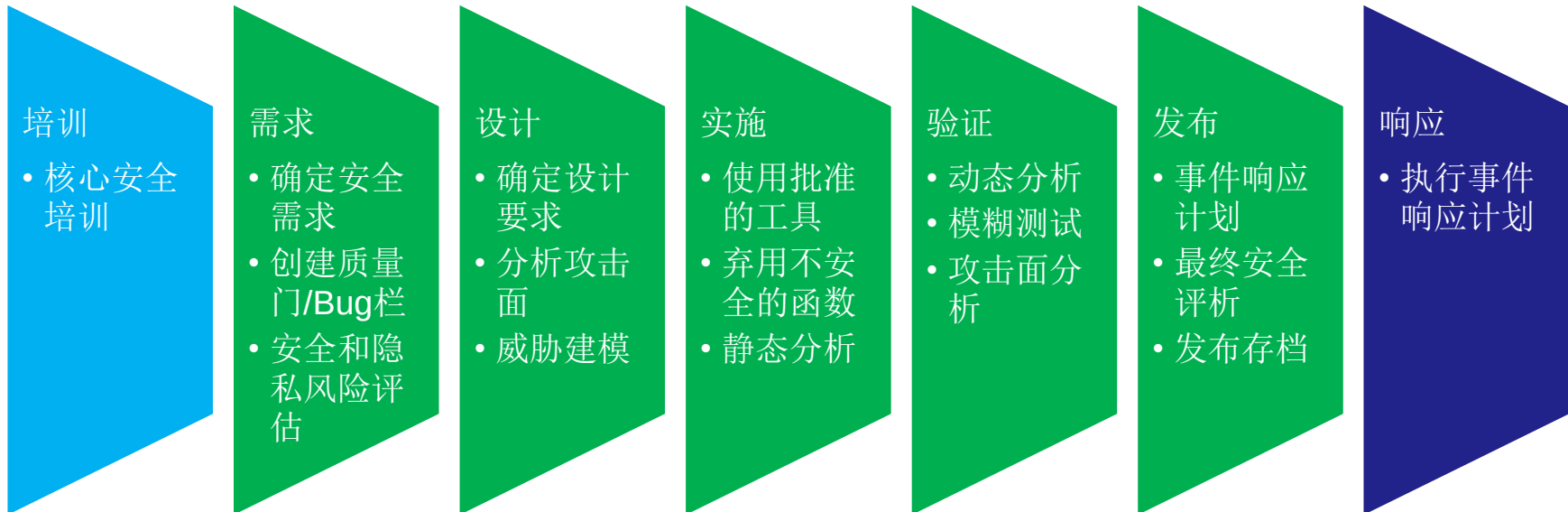
- 安全开发生命周期 SDL (The Trustworthy Computing Security Development Lifecycle)

- 自 2004 起, SDL 作为全公司的计划和强制政策, 在将安全和隐私植入软件和企业文化方面发挥了重要作用。
- 核心是将安全考虑集成在软件开发的每一个阶段, SDL 致力于减少软件中漏洞的数量和严重性。SDL 在开发过程的所有阶段中均引入了安全和隐私。

软件安全开发生命周期

SDL的阶段和安全活动

- 软件安全开发生命阶段
 - 5+2个阶段
 - 17项必需的安全活动



七个阶段的主要任务和目的

1、培训

针对开发团队和高层进行安全意识与能力的培训，使之了解安全基础知识以及安全方面的最新趋势，同时能针对新的安全问题与形势持续提升团队的能力。

2、需求

在软件开发初始阶段，确定软件安全需要遵循的安全标准和相关要求，**建立安全和隐私要求的最低可接受级别。**

3、设计

设计阶段要从安全性的角度定义软件的总体结构。通过分析攻击面，**设计相应的功能和策略，降低并减少不必要的安全风险，分析软件或系统的安全威胁，提出缓解措施**

七个阶段的主要任务和目的

4、实施

按照设计要求，对软件进行编码和集成，实现相应的安全功能、策略以及缓解措施。在该阶段通过安全编码和禁用不安全的API，可以减少实现时导致的安全问题和编码引入的安全漏洞，并通过代码静态分析等措施来确保安全编码规范的实施。

5、验证

通过动态分析和安全测试手段，检测软件的安全漏洞，全面核查攻击面，检查各个关键因素上的威胁缓解措施是否得以正确实现。

七个阶段的主要任务和目的

6、发布

建立可持续的**安全维护响应计划**，对软件进行最终安全核查。本阶段应将所有相关信息和数据存档，以便对软件进行发布与维护。这些信息和数据包括所有规范、源代码、二进制文件、专用符号、威胁模型、文档、应急响应计划等。

7、响应

响应安全事件与漏洞报告，实施漏洞修复和应急响应。同时发现新的问题与安全问题模式，并将它们用于SDL的持续改进过程中。

软件安全开发生命周期

IE采用SDL后的效果：正式发布后12个月内修复的漏洞总数

